

VULNERABILITY ASSESSMENT REPORT

Automotive CAN Bus — Rolling Code Key Fob

Engagement / Room	TryHackMe — Phantom Fob
Target Host	10.48.173.153
Assessment Type	Black-box protocol & cryptographic analysis (CAN bus)
Report Classification	TLP:AMBER — Training / Lab Environment
Overall Risk Rating	CRITICAL
Report Date	26 September 2026
Prepared For	Room Owner / Training Stakeholders
Prepared By	Security Assessment Team

Table of Contents

- 1. Executive Summary
- 2. Scope of Engagement
- 3. Methodology
- 4. Reconnaissance & Service Enumeration
- 5. Findings
 - 5.1 Forgeable Rolling-Code Authentication on Key Fob Command Bus (Critical)
- 6. Technical Analysis & Exploitation Walkthrough
- 7. Proof of Exploitation
- 8. Risk Rating & CVSS-style Scoring
- 9. Remediation Recommendations
- 10. Conclusion
- Appendix A — Nmap Scan Output
- Appendix B — Captured Frame Dataset

1. Executive Summary

This engagement assessed the security of a simulated automotive Controller Area Network (CAN) bus exposed by the **Phantom Fob** lab environment, focusing on the wireless key-fob command channel used to Lock, sound the Horn, and Arm/Disarm the vehicle immobiliser. The manufacturer's documentation for the in-scope system asserts that its unlock command "can't be copied," implying a rolling-code (time- and counter-based) authentication scheme resistant to replay and cloning attacks.

Through passive traffic analysis followed by targeted differential cryptanalysis, the assessment team fully reverse-engineered the fob's command frame format, isolated the command byte governing vehicle actions, and recovered the exact XOR-based checksum function protecting frame integrity. Using this recovered logic, the team successfully **forged a valid, never-before-transmitted Unlock command** and injected it directly onto the bus, causing the vehicle to unlock despite the key fob never having an Unlock button and despite the vendor's copy-protection claim. This constitutes a complete authentication bypass of the vehicle's remote-command security control.

This finding is rated **Critical**. In a real-world equivalent system, this class of vulnerability would allow an attacker with passive RF/bus access to unlock and potentially disable the immobiliser on any vehicle using the same key derivation logic, without possessing the physical key fob.

2. Scope of Engagement

In-scope host	10.48.173.153
In-scope services	TCP/8080 (Instrument Cluster dashboard/HTTP), TCP/29536 (raw CAN bus relay)
Out of scope	TCP/22 (SSH) — enumerated only, not tested for authentication bypass
Assessment style	Unauthenticated, black-box, passive + active bus interaction
Tooling	nmap, netcat, custom Python (socket/subprocess-based CAN client), differential cryptanalysis scripts

3. Methodology

The assessment followed a structured black-box protocol reverse-engineering methodology:

- **Reconnaissance:** Network service enumeration via Nmap to identify exposed interfaces.
- **Protocol discovery:** Interactive probing of the raw bus socket (TCP/29536) to identify the handshake sequence (<open vcan0> → <rawmode>) and streaming frame format.
- **Baseline capture:** Passive capture of idle bus traffic to fingerprint background/periodic traffic (10 recurring CAN IDs) and establish a noise floor.
- **Differential traffic analysis:** Repeated, precisely-timed activation of each available fob function (Lock, Horn, Immobiliser Arm/Disarm) via the dashboard's control API (POST /press), correlating each action with newly observed or changed bus frames to isolate the fob's dedicated CAN identifier (0x471).
- **Structural decomposition:** Byte-level comparison across many captured frames to classify each byte position as fixed, command-dependent, counter-derived, time-derived, or checksum.

- **Cryptanalysis:** Automated brute-force search across common checksum constructions (CRC-8 variants, XOR chains, additive checksums) over all byte-position subsets, matching outputs against real captured frames to recover the exact integrity-check formula.
- **Exploitation:** Construction of a forged command frame using a freshly captured (still time-valid) frame as a template, substitution of an untested command byte, recomputation of the correct checksum, and direct injection onto the bus via the relay's <send> primitive.

4. Reconnaissance & Service Enumeration

An Nmap TCP scan of the target host identified three open services:

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-26 13:05 IST
Nmap scan report for 10.48.173.153
Host is up (0.082s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   256 bf:f5:92:ad:7f:3f:14:05:c6:33:d0:72:90:51:3a:29 (ECDSA)
|_  256 6a:10:3d:04:6d:5e:b1:57:af:89:da:33:7e:04:44:e4 (ED25519)
8080/tcp  open  http-proxy
|_http-title: Instrument Cluster
29536/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 99.46 seconds
```

Port 8080 hosts a web-based "Instrument Cluster" dashboard that visualises live vehicle telemetry via Server-Sent Events (GET /events) and exposes a fob-press control API (POST /press) accepting the button values LOCK, HORN, IMMOB_ARM, and IMMOB_DISARM. Port 29536 was found to be a bespoke line-based relay directly exposing the vehicle's simulated CAN bus in near real time, and — critically — supports both passive frame streaming and **active frame injection** via a documented-in-practice send command.

5. Findings

5.1 Forgeable Rolling-Code Authentication on Key Fob Command Bus

CRITICAL

CWE-294: Authentication Bypass by Capture-replay / Predictable Value |
CWE-330: Use of Insufficiently Random Values

Description: The key fob's command channel (CAN ID 0x471) implements a rolling-code scheme intended to prevent replay and cloning. However, the integrity check protecting each frame is a simple, statically-keyed XOR checksum over four bytes of the frame itself, and the two "secret" time-derived bytes are broadcast in cleartext on every command and remain valid for a short but sufficient reuse window (multiple sequential messages). An attacker who can passively observe a single legitimate command has enough information to correctly compute a valid checksum for **any other command value**, including one never sent by the legitimate fob (e.g., an Unlock command that has no physical button).

Frame Structure (Recovered)

Byte	B0	B1	B2	B3	B4	B5	B6	B7
Field	Time-part	Fixed	Fixed	Command	Time-part	Fixed	Counter	Checksum

Observed	variable	0xFC	0x29	variable	variable	0xD6	incrementing	derived
----------	----------	------	------	----------	----------	------	--------------	---------

Recovered known command byte (B3) values:

Function	Command Byte (B3)
Lock	0x9C
Horn	0x29
Immobiliser Disarm	0x7A
Immobiliser Arm	0xE0
Unlock (undocumented / no fob button)	Recovered via forged-frame brute force — see Section 7

Recovered checksum formula: Differential analysis across twenty paired captures (same-instant Lock/Horn transmissions) combined with an automated search across CRC-8 and XOR/additive constructions established the following relationship, which held across every captured frame:

$$B7 = B0 \oplus B3 \oplus B4 \oplus B6 \oplus 0xB2$$

This is a static, key-free XOR combination of the command byte, counter byte, and the two time-derived bytes. Because B0 and B4 are transmitted in the clear on every frame and are shared across different commands issued within the same short time window, an attacker requires only **one** recent legitimate frame to compute a fully valid checksum for a forged frame carrying an arbitrary, attacker-chosen command byte.

6. Technical Analysis & Exploitation Walkthrough

The exploitation chain proceeded through the following stages:

- **Stage 1 — Protocol discovery:** Connected to TCP/29536; identified the two-step handshake (<open vcan0>, <rawmode>) required before the relay streams live <frame ID timestamp data> records.
- **Stage 2 — ID isolation:** Cross-referenced timestamped button presses (via the dashboard control API) against the live frame stream to isolate CAN ID 0x471 as the fob command channel, distinct from ten background telemetry IDs.
- **Stage 3 — Field classification:** Captured dozens of frames across all four legitimate commands and multiple repeated presses of the same command, allowing each byte to be classified as fixed, counter, command, time-derived, or checksum by observing which bytes co-varied with which input.
- **Stage 4 — Checksum recovery:** Captured near-simultaneous Lock/Horn frame pairs to hold the time-component constant while the command byte varied, isolating the checksum's dependency on the command byte, then solved the exact XOR formula via automated search.
- **Stage 5 — Forgery and injection:** Captured a fresh legitimate frame to obtain currently-valid time bytes, iterated candidate command-byte values not used by any known function, recomputed a valid checksum for each using the recovered formula, and transmitted the resulting frame directly onto the bus using the relay's <send 471 8 <bytes>> primitive, refreshing the time-derived template every few attempts to stay within its short validity window.
- **Stage 6 — Validation:** Monitored the dashboard's real-time status feed (GET /event/s) for a transition of the locked state to false and for population of the flag field, confirming successful forgery of a previously non-existent Unlock command.

7. Proof of Exploitation

A forged frame on CAN ID 0x471, carrying an untested command byte and a checksum computed with the recovered formula, was accepted by the vehicle's door control logic. The dashboard's live telemetry feed transitioned the door status from LOCKED to UNLOCKED and surfaced the challenge completion flag, confirming full compromise of the command authentication mechanism without ever possessing a physical Unlock button or capturing a genuine Unlock transmission.

Note: the literal flag value is considered engagement-sensitive proof-of-completion data and has been withheld from this report; it can be appended by the report owner if required for internal record-keeping.

8. Risk Rating & CVSS-style Scoring

Scored using CVSS v3.1 vector conventions to illustrate real-world severity if this design pattern were deployed in a production vehicle:

CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:H → Base Score: 8.8 (High/Critical in context)

Metric	Value	Justification
Attack Vector	Adjacent (A)	Requires proximity to the vehicle's wireless/bus range
Attack Complexity	Low (L)	Checksum is a static formula requiring no brute-forcing of secret keys
Privileges Required	None (N)	No prior authentication or physical key possession needed
User Interaction	None (N)	Fully automatable once formula is known
Confidentiality	None (N)	No data disclosure component
Integrity	High (H)	Attacker can command physical vehicle state (unlock)
Availability	High (H)	Could equally be used to defeat the immobiliser

9. Remediation Recommendations

- **Replace static XOR checksum with a cryptographic MAC.** Use HMAC-SHA256 (or CMAC-AES for constrained ECUs) computed with a per-vehicle secret key never transmitted on the bus, rather than a fixed-key XOR of visible fields.
- **Do not transmit time/seed material in cleartext.** If a time- or counter-based rolling factor is used, it must not be independently derivable from the command byte in the same message; consider a properly seeded HMAC-based OTP construction (e.g., HOTP/TOTP per RFC 4226/6238) rather than a reversible arithmetic combination.
- **Enforce strict counter monotonicity with replay rejection,** and reject any frame whose counter is not strictly greater than the last accepted value for that command class, closing the reuse window currently exploited between successive legitimate transmissions.
- **Bind the integrity check to the full frame including the command field** using a keyed function, so that recovering the checksum computation for one command provides no assistance in forging any other command.

- **Rate-limit and anomaly-detect on the receiving ECU.** A door control unit accepting 250+ distinct command frames within seconds from a single source is itself an indicator that should trigger a lockout or alert in a production system.
 - **Independent third-party cryptographic review** of the finalised rolling-code design prior to any production deployment, given the demonstrated gap between the vendor's "can't be copied" claim and the recovered implementation.
-

10. Conclusion

The assessed key-fob command scheme, despite superficially resembling a rolling-code design, relies on a reversible, unkeyed checksum and cleartext time-derived fields. This allowed complete recovery of the message-authentication logic through passive traffic analysis alone, and enabled the assessment team to synthesize a fully valid Unlock command that was never transmitted by the legitimate fob. The vendor's assertion that the unlock command "can't be copied" is not supported by the current implementation; the correct characterisation is that the command cannot be *replayed*, but it can be *forged* once the checksum construction is known — and that construction was recoverable within a single assessment session using only bus traffic. This finding should be treated as Critical and remediated prior to any production use of this design.

Appendix A — Nmap Scan Output

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-26 13:05 IST
Nmap scan report for 10.48.173.153
Host is up (0.082s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
| ssh-hostkey:
|   256 bf:f5:92:ad:7f:3f:14:05:c6:33:d0:72:90:51:3a:29 (ECDSA)
|_  256 6a:10:3d:04:6d:5e:b1:57:af:89:da:33:7e:04:44:e4 (ED25519)
8080/tcp  open  http-proxy
|_http-title: Instrument Cluster
29536/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 99.46 seconds
```

Appendix B — Captured Frame Dataset (CAN ID 0x471)

Representative same-instant Lock/Horn pairs used to isolate the checksum construction:

83FC29290FD69F88	(HORN)	83FC299C0FD6A002	(LOCK)
B8FC299C27D6A110	(LOCK)	B8FC292927D6A2A6	(HORN)
7DFC299CADD6A35D	(LOCK)	7DFC2929ADD6A4EF	(HORN)
D3FC299CAAD6A5F2	(LOCK)	D3FC2929AAD6A644	(HORN)
1FFC292930D6A713	(HORN)	1FFC299C30D6A8A9	(LOCK)
6CFC299C3FD6A9D4	(LOCK)	6CFC29293FD6AA62	(HORN)
85FC292946D6ABF3	(HORN)	85FC299C46D6AC41	(LOCK)
3AFC2929F7D6ADFB	(HORN)	3AFC299CF7D6AE4D	(LOCK)
4CFC299C75D6AFB8	(LOCK)	4CFC292975D6B012	(HORN)
E1FC29293BD6B1F0	(HORN)	E1FC299C3BD6B246	(LOCK)

Byte layout confirmed across the dataset: B0 and B4 are identical within each same-instant pair (time-derived, command-independent); B3 differs and matches the known command byte for each function; B6 increments by exactly 1 between paired messages (counter); B7 satisfies $B0 \oplus B3 \oplus B4 \oplus B6 \oplus 0xB2$ in every row.